

# Sentiora Development & Implementation Plan v1.0

---

v1.0 — Engineering Execution Roadmap

Sentiora Engineering

August 2026 – December 2026

1. Project Overview
  - 1.1 Purpose of This Document
  - 1.2 Objectives
  - 1.3 Timeline Summary
  - 1.4 Success Criteria
  - 1.5 Definition of MVP Completion
2. Development Principles
  - 2.1 Code Quality
  - 2.2 Git Workflow
  - 2.3 Commit Message Format
  - 2.4 Code Review
  - 2.5 Naming Conventions
  - 2.6 Definition of Ready (DoR)
  - 2.7 Definition of Done (DoD)
3. Recommended Repository Structure
  - Folder purpose summary
4. Environment Setup
  - 4.1 Required Software
    - Recommended VS Code Extensions
  - 4.2 Environment Variables ( `.env.example` )
  - 4.3 Local Setup
  - 4.4 Running Locally (Full Stack)
  - 4.5 Production Setup
5. Development Phases
  - Phase 0 — Environment & Foundations (Week 1, August)
  - Phase 1 — Authentication (Weeks 2-3, August)
  - Phase 2 — Chrome Extension Skeleton (Weeks 3-4, August)
  - Phase 3 — Capture Pipeline (Weeks 5-7, September)
  - Phase 4 — Backend Core APIs (Weeks 5-8, September, parallel to Phase 3)
  - Phase 5 — Frontend Dashboard (Weeks 8-10, October)
  - Phase 6 — Search (Weeks 10-12, October-November)
  - Phase 7 — AI / RAG Chat (Weeks 12-15, November)
  - Phase 8 — Testing (Weeks 15-17, November-December)
  - Phase 9 — Deployment & Launch (Weeks 17-18, December)
6. Sprint Planning
7. Module-wise Development Order
8. Backend Development Order
9. Frontend Development Order
10. Chrome Extension Development Order
11. Integration Plan
12. Testing Strategy
13. Bug Management
  - 13.1 Severity Levels

|      |                                        |
|------|----------------------------------------|
| 13.2 | Priority Levels                        |
| 13.3 | Issue Lifecycle                        |
| 13.4 | GitHub Issues Workflow                 |
| 13.5 | Labels                                 |
| 13.6 | Milestones                             |
| 14.  | Risk Assessment                        |
| 15.  | CI/CD Plan                             |
| 15.1 | GitHub Actions Pipelines               |
| 15.2 | Preview Deployments                    |
| 15.3 | Production Deployment                  |
| 15.4 | Rollback Strategy                      |
| 16.  | Versioning Strategy                    |
| 17.  | Documentation Strategy                 |
| 18.  | Deployment Roadmap                     |
| 19.  | Launch Checklist                       |
| 20.  | Post-MVP Roadmap                       |
| 21.  | Month-by-Month Roadmap                 |
|      | August — Foundations                   |
|      | September — Core Data Loop             |
|      | October — Visibility & Discoverability |
|      | November — Intelligence Layer          |
|      | December — Hardening & Launch          |
|      | Appendix: Document Control             |

# 1. Project Overview

---

## 1.1 Purpose of This Document

---

This is **not** the PRD, SRS, UI/UX Specification, Backend Technical Specification, API Specification, or Database Schema. Those artifacts are complete and frozen. This document is the **engineering execution roadmap**: the day-to-day guide that tells the development team *how* Sentiora gets built, in what order, on what cadence, with what quality bar, and how it ships.

If the PRD answers “what are we building and why,” this plan answers “**how will Sentiora actually be built?**”

## 1.2 Objectives

---

- Translate the locked product/technical scope into an executable, week-by-week engineering plan.
- Give every developer a clear answer to “what do I build next, and what does it depend on?”
- Establish engineering standards (Git workflow, code review, DoD/DoR) so a 4-person student team ships consistently.
- Define testing, CI/CD, versioning, and deployment practices appropriate for a lean team shipping a real product.
- Separate MVP scope from Post-MVP scope with zero ambiguity.

## 1.3 Timeline Summary

---

| Period | Focus |
|--------|-------|
|--------|-------|

|           |                                              |
|-----------|----------------------------------------------|
| August    | Environment, repo, auth, extension skeleton  |
| September | Capture pipeline, backend core APIs, storage |
| October   | Memory, search, embeddings, dashboard        |
| November  | RAG chat, AI features, integration hardening |
| December  | Testing, CI/CD, deployment, launch           |

Total duration: **5 months** (August → December), MVP target: **early December**.

## 1.4 Success Criteria

The project is considered successful when:

1. A user can install the Chrome Extension, authenticate, and grant per-source capture permissions.
2. The Meaningful Capture Engine captures webpage, PDF, and YouTube content the user has approved, and never captures excluded sensitive categories (passwords, payment, banking, health, incognito).
3. Captured content is visible in a Memory Timeline in the Web Dashboard.
4. Semantic Search returns relevant results across a user's captured Memory.
5. RAG Chat answers questions grounded in Memory content with source citations.
6. The system is deployed to a staging and production environment with monitoring, logging, and backups in place.
7. All P0/P1 bugs from Testing phase are resolved prior to launch.

## 1.5 Definition of MVP Completion

MVP is “done” when every item in the **Launch Checklist (Section 19)** is checked, the app is live in production, and a real user (dogfooding: Vansh, a student/self-learner) can complete the full loop — **capture → memory → search → RAG chat with citations** — end to end without engineering intervention.

Explicitly **out of scope for MVP**: Footprint module, Shield module, Firefox/Edge extensions, mobile app, team collaboration, streaming AI responses. See Section 20.

# 2. Development Principles

## 2.1 Code Quality

- **Modular architecture**: every feature is a self-contained module (route + service + schema + tests) — no cross-module reach-ins.
- **Clean code**: small functions, descriptive names, no dead code, no commented-out blocks committed to `main`.
- **Type safety**: TypeScript `strict: true` on frontend/extension; Python type hints + `mypy` (or `pyright`) on backend.
- **Documentation**: every module has a top-of-file docstring/comment explaining purpose; every public function has a docstring.
- **No magic numbers/strings**: constants live in a shared `constants` file per app.

## 2.2 Git Workflow

**Branching model:** trunk-based with short-lived feature branches (GitHub Flow — appropriate for a 4-person team; avoids GitFlow overhead).

```
main                → always deployable, protected branch
├─ feature/xxx      → short-lived, 1–3 days max
├─ fix/xxx
└─ chore/xxx
```

Rules: - `main` is protected: no direct pushes, requires 1 approving review + passing CI. - Branch names: `feature/<module>-<short-desc>` (e.g. `feature/capture-engine-dwell-filter`). - Rebase (not merge commits) onto `main` before opening a PR when possible. - Delete branches after merge.

## 2.3 Commit Message Format

Conventional Commits:

```
<type>(<scope>): <short summary>

[optional body]
[optional footer: Closes #123]
```

Types: `feat`, `fix`, `chore`, `refactor`, `test`, `docs`, `perf`, `ci`.

Examples:

```
feat(extension): add dwell-time tracking to content script
fix(search): correct pgvector cosine distance ordering
docs(readme): update local setup instructions
```

## 2.4 Code Review

- Every PR requires **at least 1 approval** before merge (4-person team → rotate reviewers).
- Reviewer checklist: correctness, test coverage, naming, no secrets committed, matches DoD.
- PRs should be **small** (< 400 lines diff where possible) — split large features into incremental PRs behind feature flags if needed.
- Author responds to comments within 1 business day; reviewer reviews within 1 business day.

## 2.5 Naming Conventions

| Context              | Convention               | Example                              |
|----------------------|--------------------------|--------------------------------------|
| React components     | PascalCase               | <code>MemoryTimeline.tsx</code>      |
| React hooks          | camelCase, use prefix    | <code>useCaptureSettings.ts</code>   |
| Python modules/files | snake_case               | <code>capture_service.py</code>      |
| Python classes       | PascalCase               | <code>CaptureEngine</code>           |
| DB tables            | snake_case, plural       | <code>memory_items</code>            |
| API routes           | kebab-case, plural nouns | <code>/api/v1/memory-items</code>    |
| Env vars             | UPPER_SNAKE_CASE         | <code>DATABASE_URL</code>            |
| Git branches         | kebab-case               | <code>feature/rag-citation-ui</code> |

## 2.6 Definition of Ready (DoR)

A ticket is “ready” to be picked up when:

- It references the relevant PRD/spec section (no re-litigating scope in-ticket).
- Acceptance criteria are written as checkable bullets.
- Dependencies (API contracts, schema fields) are identified and available or stubbed.
- Design/UX reference exists for any UI-facing ticket.

## 2.7 Definition of Done (DoD)

A ticket is “done” when:

- Code merged to `main` via reviewed PR.
- Unit tests written and passing for new logic.
- No new linter/type errors.
- Manually verified against acceptance criteria.
- Documentation (README/API docs) updated if behavior changed.
- No known P0/P1 regressions introduced.

## 3. Recommended Repository Structure

Monorepo, managed with a workspace tool (npm/pnpm workspaces for JS packages; Python backend as its own top-level package).

```
sentiora/
├── frontend/                                # React + TS web dashboard
│   ├── src/
│   │   ├── app/                            # routes (React Router)
│   │   ├── components/                    # shared UI components
│   │   ├── features/                      # feature-sliced modules (memory, search, ai, settings...)
│   │   ├── hooks/
│   │   ├── stores/                        # Zustand stores
│   │   ├── lib/                          # api client, query client, utils
│   │   ├── types/
│   │   └── styles/
│   ├── public/
│   ├── index.html
│   ├── vite.config.ts
│   └── package.json
├── extension/                              # Chrome Extension (Manifest V3)
│   ├── src/
│   │   ├── popup/                        # popup UI (React)
│   │   ├── background/                  # service worker
│   │   ├── content-scripts/             # page-injected capture scripts
│   │   ├── capture-engine/              # meaningful capture logic (dwell time, filters)
│   │   ├── messaging/                  # cross-context message bus
│   │   └── permissions/                 # consent + per-source toggle logic
│   ├── manifest.json
│   └── package.json
├── backend/                               # FastAPI application
│   ├── app/
│   │   ├── api/v1/                      # route modules (auth, memory, search, ai, users...)
│   │   ├── core/                       # config, security, dependencies
│   │   ├── models/                     # SQLAlchemy models
│   │   ├── schemas/                    # Pydantic schemas
│   │   ├── services/                   # business logic per module
│   │   ├── workers/                   # Celery task definitions
│   │   ├── rag/                        # LangChain pipelines, embeddings, retrieval
│   │   ├── db/                         # session, migrations (Alembic)
│   │   └── main.py
│   ├── tests/
│   ├── alembic/
│   ├── requirements.txt / pyproject.toml
│   └── Dockerfile
├── shared/                                # cross-package shared types/constants
│   ├── types/                           # OpenAPI-generated or hand-shared TS types
│   └── constants/
└── infrastructure/                       # IaC, deployment configs
```

```

├── docker/
├── railway/ or aws/
├── nginx/ (if applicable)

├── scripts/                # dev automation
│   ├── setup.sh
│   ├── seed_db.py
│   └── generate_types.sh

├── tests/                  # end-to-end tests (Playwright) spanning frontend+extension+backend
│   └── e2e/

├── docs/                  # engineering docs (this plan lives here)
│   ├── architecture/
│   ├── setup-guide.md
│   ├── contribution-guide.md
│   └── changelog.md

├── .github/
│   └── workflows/         # CI/CD pipelines

├── docker-compose.yml     # local dev: postgres, redis, minio, backend, frontend
├── .env.example
└── README.md

```

### Folder purpose summary

| Folder             | Purpose                                                            |
|--------------------|--------------------------------------------------------------------|
| frontend/          | Web Dashboard — Memory Timeline, Search, AI Chat, Settings UI      |
| extension/         | Chrome Extension — capture engine, consent UI, background sync     |
| backend/           | FastAPI services — auth, memory CRUD, search, RAG, workers         |
| shared/            | Types/constants shared across frontend, extension, backend clients |
| infrastructure/    | Docker, deployment manifests, reverse proxy config                 |
| scripts/           | One-off and recurring dev-automation scripts                       |
| tests/             | Cross-cutting end-to-end tests                                     |
| docs/              | Living engineering documentation                                   |
| .github/workflows/ | CI/CD pipeline definitions                                         |

## 4. Environment Setup

### 4.1 Required Software

| Tool                    | Version  | Purpose                                 |
|-------------------------|----------|-----------------------------------------|
| Node.js                 | 20.x LTS | Frontend + Extension builds             |
| npm / pnpm              | pnpm 9.x | Package management (workspaces)         |
| Python                  | 3.11+    | Backend                                 |
| Docker + Docker Compose | latest   | Local services (Postgres, Redis, MinIO) |
|                         |          |                                         |

|                          |        |                 |
|--------------------------|--------|-----------------|
| Git                      | latest | Version control |
| PostgreSQL client (psql) | 15+    | DB inspection   |
| VS Code                  | latest | Recommended IDE |

## Recommended VS Code Extensions

- ESLint, Prettier
- Python, Pylance
- Docker
- GitLens
- Tailwind CSS IntelliSense
- Thunder Client / REST Client (API testing)

## 4.2 Environment Variables ( `.env.example` )

```
# Backend
DATABASE_URL=postgresql://sentiora:sentiora@localhost:5432/sentiora
REDIS_URL=redis://localhost:6379/0
SECRET_KEY=changeme
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60

# Storage
S3_ENDPOINT=http://localhost:9000
S3_ACCESS_KEY=minioadmin
S3_SECRET_KEY=minioadmin
S3_BUCKET=sentiora-dev

# AI
LLM_API_BASE=https://api.openai.com/v1
LLM_API_KEY=changeme
EMBEDDING_MODEL=text-embedding-3-small

# Frontend
VITE_API_BASE_URL=http://localhost:8000/api/v1

# Extension
EXTENSION_API_BASE_URL=http://localhost:8000/api/v1
```

## 4.3 Local Setup

```
git clone <repo-url>
cd sentiora
cp .env.example .env

# Start infra (Postgres, Redis, MinIO)
docker compose up -d postgres redis minio

# Backend
cd backend
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
alembic upgrade head
uvicorn app.main:app --reload --port 8000

# Frontend
cd ../frontend
pnpm install
pnpm dev

# Extension
cd ../extension
```

```
pnpm install
pnpm build:dev
# then load /extension/dist as an unpacked extension in chrome://extensions
```

## 4.4 Running Locally (Full Stack)

```
docker compose up -d          # infra
docker compose up backend     # or run uvicorn directly for hot reload
pnpm --filter frontend dev
pnpm --filter extension build:dev --watch
```

RQ worker (for capture processing / embedding jobs):

```
python -m app.workers.worker
```

## 4.5 Production Setup

- Backend containerized via Docker, deployed to Railway/Render/AWS.
- Frontend deployed to Vercel (static build + edge functions if needed).
- PostgreSQL managed instance with pgvector extension enabled.
- Redis managed instance (Celery broker + result backend).
- MinIO/S3 for object storage (captured page snapshots, PDFs).
- Secrets managed via platform's secret manager (Railway/Render env vars or AWS Secrets Manager) — **never committed**.

# 5. Development Phases

Each phase lists objectives, deliverables, dependencies, exit criteria, and estimated duration.

## Phase 0 — Environment & Foundations (Week 1, August)

- **Objectives:** repo scaffolding, CI skeleton, local dev environment working for all 4 devs.
- **Deliverables:** monorepo structure, docker-compose, `.env.example`, base CI pipeline (lint+build).
- **Dependencies:** none.
- **Exit Criteria:** every team member can run backend + frontend + extension locally from a clean clone in under 30 minutes.
- **Duration:** 1 week.

## Phase 1 — Authentication (Weeks 2-3, August)

- **Objectives:** user signup/login, JWT session management, protected routes.
- **Deliverables:** `/auth/register`, `/auth/login`, `/auth/refresh` endpoints; frontend auth flow; extension auth flow (token storage in `chrome.storage`).
- **Dependencies:** Phase 0 complete; DB schema for `users` (per locked DB schema doc).
- **Exit Criteria:** a user can register, log in on web, and the extension recognizes the same session.
- **Duration:** 2 weeks.

## Phase 2 — Chrome Extension Skeleton (Weeks 3-4, August)



- **Objectives:** Manifest V3 skeleton, popup shell, background service worker, permissions model.
- **Deliverables:** installable unpacked extension with popup UI, login state, per-source consent toggles (UI only, no capture logic yet).
- **Dependencies:** Phase 1 (auth).
- **Exit Criteria:** extension installs, authenticates, and displays consent toggle UI persisted to backend.
- **Duration:** 2 weeks (overlaps Phase 1 tail).

## Phase 3 — Capture Pipeline (Weeks 5-7, September)

---

- **Objectives:** implement the Meaningful Capture Engine — content scripts, dwell-time/engagement filtering, never-capture exclusion rules.
- **Deliverables:** content script capturing webpage text/metadata; PDF capture path; YouTube capture path; manual save; sensitive-content exclusion (passwords/payment/banking/health/incognito) enforced client-side before any network call.
- **Dependencies:** Phase 2 (extension skeleton).
- **Exit Criteria:** browsing a real session produces correctly filtered capture events queued for sync; excluded categories verified to never leave the browser.
- **Duration:** 3 weeks.

## Phase 4 — Backend Core APIs (Weeks 5-8, September, parallel to Phase 3)

---

- **Objectives:** Memory CRUD APIs, ingestion endpoint, storage integration (S3/MinIO), Celery task queue wiring.
- **Deliverables:** `/memory-items` CRUD, `/capture/ingest`, file storage service, Celery worker skeleton processing ingestion jobs asynchronously.
- **Dependencies:** Phase 0; DB schema.
- **Exit Criteria:** extension can push a captured item and retrieve it back via API; async job visibly processes it (status transitions).
- **Duration:** 4 weeks.

## Phase 5 — Frontend Dashboard (Weeks 8-10, October)

---

- **Objectives:** Memory Timeline UI, source filters, item detail view, settings pages.
- **Deliverables:** dashboard routes (`/memory`, `/settings`), timeline component with infinite scroll, per-source consent management UI (mirrors extension state).
- **Dependencies:** Phase 4 APIs available.
- **Exit Criteria:** a captured item flows extension → backend → dashboard visibly within seconds.
- **Duration:** 3 weeks.

## Phase 6 — Search (Weeks 10-12, October-November)

---

- **Objectives:** embedding generation pipeline, pgvector-backed semantic search, search UI.
- **Deliverables:** embedding Celery task triggered on ingestion; `/search` endpoint (vector similarity + optional keyword fallback); search UI in dashboard with result snippets and source links.
- **Dependencies:** Phase 4 (storage), Phase 5 (dashboard shell).
- **Exit Criteria:** querying returns relevant memory items ranked by semantic similarity, sub-2s p50 latency locally.
- **Duration:** 3 weeks.

## Phase 7 — AI / RAG Chat (Weeks 12-15, November)

- **Objectives:** RAG pipeline (LangChain), chat endpoint, citation mapping, chat UI.
- **Deliverables:** `/ai/chat` endpoint retrieving top-k memory chunks, composing grounded prompt, returning answer + source citations; chat UI in dashboard with citation chips linking back to source memory items.
- **Dependencies:** Phase 6 (search/embeddings).
- **Exit Criteria:** a chat question about the user's own captured content returns a grounded answer with correct, clickable citations.
- **Duration:** 3-4 weeks.

## Phase 8 — Testing (Weeks 15-17, November-December)

- **Objectives:** close test coverage gaps, E2E test suite, security pass, performance pass.
- **Deliverables:** unit/integration/E2E suites green in CI; security checklist complete; load-test results for search/chat endpoints.
- **Dependencies:** Phases 1-7 functionally complete.
- **Exit Criteria:** no open P0/P1 bugs; CI green on `main`.
- **Duration:** 2-3 weeks.

## Phase 9 — Deployment & Launch (Weeks 17-18, December)

- **Objectives:** staging + production deployment, monitoring/logging/backups, launch checklist sign-off.
- **Deliverables:** production environment live; Chrome Web Store submission prepared (or side-load package for MVP dogfood); monitoring dashboards.
- **Dependencies:** Phase 8 complete.
- **Exit Criteria:** Section 19 Launch Checklist fully checked.
- **Duration:** 1-2 weeks.

## 6. Sprint Planning

2-week sprints, realistic for a 4-person student team (part-time capacity assumed alongside coursework).

| Sprint   | Dates (approx.) | Goals & Features                                                   | Deliverables                        | Expected Outcome                     |
|----------|-----------------|--------------------------------------------------------------------|-------------------------------------|--------------------------------------|
| Sprint 0 | Aug W1-2        | Repo, CI, docker-compose, environment parity                       | Working local env for all devs      | Team unblocked to start feature work |
| Sprint 1 | Aug W3-4        | Auth (backend+frontend+extension), extension skeleton              | Login works across web + extension  | Shared identity across surfaces      |
| Sprint 2 | Sep W1-2        | Capture engine v1 (webpage + manual save), never-capture rules     | Filtered captures queued locally    | Core differentiator functional       |
| Sprint 3 | Sep W3-4        | Backend ingestion API, storage, Celery wiring, PDF/YouTube capture | End-to-end capture → storage        | First full vertical slice            |
| Sprint 4 | Oct W1-2        | Memory Timeline UI, settings/consent UI                            | Dashboard shows real captured items | Visible product for the first time   |

|          |          |                                                    |                                     |                            |
|----------|----------|----------------------------------------------------|-------------------------------------|----------------------------|
| Sprint 5 | Oct W3-4 | Embedding pipeline, pgvector search backend        | /search returns real results        | Search proven on real data |
| Sprint 6 | Nov W1-2 | Search UI, RAG pipeline v1                         | Chat answers with citations (rough) | AI value prop demonstrable |
| Sprint 7 | Nov W3-4 | RAG polish, citation UI, integration hardening     | Stable end-to-end loop              | Feature-complete MVP       |
| Sprint 8 | Dec W1-2 | Testing (unit/integration/E2E), security/perf pass | Green CI, bug backlog cleared       | Launch-ready build         |
| Sprint 9 | Dec W3   | Deployment, monitoring, launch checklist           | Production live                     | <b>MVP Launched</b>        |

Each sprint: Monday planning (goals + ticket breakdown), mid-sprint check-in, Friday demo + retro.

## 7. Module-wise Development Order

| Order | Module               | Why This Order                                                                                                      |
|-------|----------------------|---------------------------------------------------------------------------------------------------------------------|
| 1     | Authentication       | Every other module needs a user identity; nothing can be tested meaningfully without it.                            |
| 2     | Users                | Profile/settings scaffolding rides along with auth; needed for per-user consent state.                              |
| 3     | Extension (skeleton) | Capture cannot exist without an installable extension shell and permission model.                                   |
| 4     | Capture              | Core differentiator (Meaningful Capture Engine) — must exist before there is any Memory to show.                    |
| 5     | Memory               | The storage/CRUD layer that Capture writes into and Dashboard reads from.                                           |
| 6     | Upload               | Manual save / PDF upload extends Memory ingestion paths once the core model is stable.                              |
| 7     | Search               | Needs a non-trivial body of Memory content to be meaningful and testable.                                           |
| 8     | AI (RAG)             | Depends on Search's embeddings/retrieval; the highest-risk, most dependent module — built last among core features. |
| 9     | Collections          | Organizational layer on top of stable Memory — low risk, can slot in once Memory is solid.                          |
| 10    | Tags                 | Similar to Collections; lightweight metadata layer.                                                                 |
| 11    | Dashboard (polish)   | Iterates continuously, but final visual/UX polish comes once underlying data (Memory, Search, AI) is stable.        |
| 12    | Settings             | Consent/privacy settings are built early (tied to Capture); general                                                 |

|    |               |                                                                           |
|----|---------------|---------------------------------------------------------------------------|
|    |               | account settings finalized last.                                          |
| 13 | Notifications | Nice-to-have, lowest risk, zero other modules depend on it — safely last. |

**Rationale in one line:** build identity → build the thing that creates data (Capture) → build the thing that stores it (Memory) → build the things that make it useful (Search, AI) → build the things that make it pleasant (Collections/Tags/Notifications).

## 8. Backend Development Order

---

1. **Database first** — schema (per locked DB Schema doc) and Alembic migrations must exist before any endpoint is written; every service depends on stable models.
2. **Authentication first** — JWT issuance/verification middleware wraps every other route; build and lock this before writing feature endpoints so they inherit auth for free.
3. **Queue first (Celery + Redis)** — capture ingestion, embedding generation, and RAG retrieval are all async by nature; standing up the worker infrastructure early avoids retrofitting async patterns into synchronous code later.
4. **Which APIs first:** `/auth` → `/users` → `/memory-items` (CRUD) → `/capture/ingest` → `/search` → `/ai/chat`. This mirrors the module order in Section 7 and ensures each API layer has real data from the prior layer to work with.
5. **Workers first (within a feature):** for both Search and AI, the background job (embedding generation, retrieval+generation) is built and tested via direct invocation/CLI *before* the HTTP endpoint wraps it — isolates business logic from web-layer concerns.
6. **Embedding pipeline:** triggered as a Celery task on successful ingestion; must be idempotent (safe to re-run if it fails partway) and chunking-aware (per RAG spec).
7. **Search:** built directly on top of the embedding pipeline's output table — cannot start until embeddings are reliably populated for at least one full capture path (webpage).
8. **AI (RAG):** last, because it composes Search (retrieval) + an external LLM call + citation mapping back to Memory — the highest number of upstream dependencies of any module.

**Reasoning summary:** backend order follows the data lifecycle — schema → auth → async infrastructure → write path (capture/ingest) → read path (CRUD) → intelligence layer (search → AI). Building intelligence before the write/read paths are stable would mean testing against fake data, which hides real integration bugs.

## 9. Frontend Development Order

---

1. **Authentication** — login/register/session screens; nothing else in the dashboard is reachable without this.
2. **Dashboard (shell)** — layout, navigation, protected route wrapper; the frame every feature screen lives inside.
3. **Memory** — Memory Timeline is the first real data screen; validates the API client, React Query caching, and data-fetching patterns the rest of the app reuses.
4. **Search** — reuses Memory's item-rendering components; adds query input + results list.
5. **AI (Chat)** — reuses Search's result/citation rendering; adds chat-specific UI (message list, streaming/loading states).
6. **Settings** — consent toggles (mirrors extension), account settings, built once the data model they control (capture sources) is stable.
7. **Responsive design** — pass applied incrementally per screen as it's built, then a dedicated responsive QA pass in Testing phase.

8. **Loading states** — skeleton loaders introduced alongside each screen’s initial build (not retrofitted).
9. **Error handling** — global error boundary + toast system built early (Phase 1), reused by every subsequent screen.

## 10. Chrome Extension Development Order

1. **Popup** — the entry point users see; built first as a static shell to unblock UI work.
2. **Authentication** — token storage in `chrome.storage.local`, login screen inside popup, session refresh logic.
3. **Permissions** — Manifest V3 `permissions / host_permissions` declarations and the runtime consent UI (per-source toggles) — must exist before any content script can be granted access.
4. **Content Script** — page-level script that reads DOM/page content; gated entirely behind the Permissions step.
5. **Background Service Worker** — coordinates content scripts, manages the capture queue, and owns network calls to the backend (service workers, not content scripts, should talk to the API for security/CSP reasons).
6. **Capture Engine** — the meaningful-capture logic (dwell time, engagement scoring, never-capture exclusion filters) layered on top of the content script + background worker plumbing.
7. **Communication** — the message-passing contract between popup ↔ content script ↔ background worker (using `chrome.runtime.sendMessage / onMessage`), formalized once all three contexts exist.
8. **Sync** — background worker batches and syncs queued captures to the backend, with retry/backoff on failure.
9. **Testing** — manual test matrix across target sites (webpage, PDF-in-browser, YouTube) plus automated tests where feasible (Jest for logic, Puppeteer for extension E2E).
10. **Publishing** — Chrome Web Store listing prep, privacy disclosure (critical given the permission-based data model), packaging, and submission review.

## 11. Integration Plan

| Integration Point   | Mechanism                                                                                                                             | Notes                                                                                     |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Backend ↔ Frontend  | REST over HTTPS, JSON, JWT bearer auth, React Query for caching/invalidation                                                          | OpenAPI schema generated from FastAPI drives shared TS types in <code>shared/types</code> |
| Extension ↔ Backend | REST over HTTPS from the background service worker only                                                                               | Content scripts never call the network directly — reduces CSP/security surface            |
| Frontend ↔ AI       | Frontend calls backend <code>/ai/chat</code> ; backend orchestrates LangChain/LLM — frontend never talks to the LLM provider directly | Keeps API keys server-side only                                                           |
| Background Workers  | Celery tasks triggered by FastAPI route handlers ( <code>.delay()</code> / <code>.apply_async()</code> )                              | Ingestion, embedding generation, RAG retrieval-heavy steps run async                      |
| Storage             | Backend service layer abstracts S3/MinIO — no direct client access from frontend/extension                                            | Signed URLs issued by backend when direct client upload/download is needed                |
| Search              | pgvector queries executed inside backend service layer, exposed via                                                                   | No direct DB access from any client                                                       |

|           |                                                                                             |                                                                                                                         |
|-----------|---------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
|           | /search                                                                                     |                                                                                                                         |
| RAG       | LangChain pipeline: retrieval (pgvector) → prompt composition → LLM call → citation mapping | Runs inside a Celery task or directly in the request handler depending on latency budget (decided during Phase 7 build) |
| Embedding | Celery task subscribed to ingestion completion events                                       | Chunking strategy per RAG/Backend Technical Spec                                                                        |

**Integration testing cadence:** after each phase in Section 5, a short integration checkpoint verifies the newly built module against its real upstream/downstream dependency (not mocks) before the next phase begins.

## 12. Testing Strategy

| Test Type           | Scope                                                                                 | Tooling                                                                                 |
|---------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Unit Testing        | Individual functions/services (backend services, React hooks, capture-engine filters) | pytest (backend), Vitest (frontend/extension)                                           |
| Integration Testing | API route + DB + service layer together                                               | pytest + test DB (Dockerized Postgres)                                                  |
| API Testing         | Contract correctness, status codes, auth enforcement                                  | pytest + httpx / Postman collection                                                     |
| Frontend Testing    | Component rendering, user interaction                                                 | React Testing Library + Vitest                                                          |
| Extension Testing   | Content script behavior, message passing, permission gating                           | Vitest (logic), manual matrix (browser behavior)                                        |
| End-to-End Testing  | Full user journeys across extension + backend + dashboard                             | Playwright                                                                              |
| Manual Testing      | Exploratory, edge cases, real-site capture verification                               | Structured test matrix, tracked in GitHub Issues                                        |
| Regression Testing  | Re-run full suite before each release tag                                             | CI on every PR + nightly on main                                                        |
| Security Testing    | Auth bypass attempts, never-capture rule verification, dependency vuln scan           | manual pentest checklist + <code>pip-audit</code> / <code>npm audit</code> + Dependabot |
| Performance Testing | Search latency, RAG response time, ingestion throughput under load                    | k6 or Locust against staging                                                            |

**Coverage targets:** ≥70% unit coverage on backend services and capture-engine logic (the two areas with the highest correctness risk); E2E suite covers the golden path (capture → memory → search → chat) plus consent/never-capture enforcement.

## 13. Bug Management

### 13.1 Severity Levels

| Severity      | Definition                            | Example                            |
|---------------|---------------------------------------|------------------------------------|
| S0 — Critical | Data loss, security breach, sensitive | Password field captured and synced |

|             |                                         |                                           |
|-------------|-----------------------------------------|-------------------------------------------|
|             | content captured despite exclusion rule |                                           |
| S1 — High   | Core flow broken for all users          | Capture pipeline fails silently           |
| S2 — Medium | Feature degraded but workaround exists  | Search returns stale results occasionally |
| S3 — Low    | Cosmetic, minor UX issue                | Misaligned button on settings page        |

## 13.2 Priority Levels

| Priority | SLA (student-team realistic)           |
|----------|----------------------------------------|
| P0       | Fix immediately, blocks all other work |
| P1       | Fix within current sprint              |
| P2       | Scheduled into next sprint             |
| P3       | Backlog, fix when convenient           |

## 13.3 Issue Lifecycle

Open → Triaged → In Progress → In Review → Resolved → Closed  
 ↓  
 Reopened (if verification fails)

## 13.4 GitHub Issues Workflow

- Every bug filed as a GitHub Issue using a `bug_report` template (steps to reproduce, expected vs actual, severity/priority labels).
- Every feature filed as a GitHub Issue using a `feature_request / task` template linked to its Phase/Sprint milestone.
- PRs reference issues via `Closes #123` to auto-close on merge.

## 13.5 Labels

`bug`, `feature`, `chore`, `severity:S0..S3`, `priority:P0..P3`, `module:capture`, `module:search`, `module:ai`, `module:extension`, `module:dashboard`, `needs-triage`, `blocked`.

## 13.6 Milestones

One GitHub Milestone per Phase (Section 5) — e.g. `Phase 3: Capture Pipeline` — with all related issues attached, giving a live burn-down per phase.

# 14. Risk Assessment

| Risk                                                       | Category  | Likelihood | Impact | Mitigation                                                                     | Contingency                          |
|------------------------------------------------------------|-----------|------------|--------|--------------------------------------------------------------------------------|--------------------------------------|
| Chrome Manifest V3 service worker lifecycle causes dropped | Technical | Medium     | High   | Persist queue to <code>chrome.storage</code> before any async work; never hold | Add retry-on-wake reconciliation job |

|                                                                |           |                 |                                     |                                                                                               |                                                                       |
|----------------------------------------------------------------|-----------|-----------------|-------------------------------------|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| capture events                                                 |           |                 |                                     | state only in memory                                                                          |                                                                       |
| pgvector query performance degrades at scale                   | Technical | Low (MVP scale) | Medium                              | Index tuning (IVFFlat/HNSW), cap corpus size for MVP                                          | Add caching layer, revisit index type                                 |
| LLM API cost/rate limits during heavy dogfooding               | Technical | Medium          | Medium                              | Cache repeated queries, cap context size                                                      | Switch to smaller/cheaper model for dev                               |
| Never-capture rule fails to exclude a sensitive page correctly | Security  | Low             | Critical                            | Explicit denylist + heuristics tested against a curated site list before any release          | Kill-switch to disable capture entirely per-source instantly          |
| 4-person student team velocity slips due to coursework         | Schedule  | High            | Medium                              | Buffer weeks built into Nov/Dec; MVP scope kept minimal (Footprint/Shield deferred)           | Cut Post-MVP-adjacent polish, not core loop                           |
| Scope creep (Footprint/Shield features requested early)        | Project   | Medium          | Medium                              | Section 20 explicitly separates Post-MVP; PRD is frozen                                       | Re-anchor to frozen PRD in scope discussions                          |
| Chrome Web Store review delays                                 | Schedule  | Medium          | Low (MVP can side-load for dogfood) | Submit early with side-load fallback plan for internal testing                                | Ship as unpacked/dev-mode extension for MVP demo if review is pending |
| Key team member unavailability                                 | Schedule  | Medium          | Medium                              | Module ownership documented, no single-owner bus factor of 1 on critical path (auth, capture) | Pair rotation on critical modules                                     |

## 15. CI/CD Plan

### 15.1 GitHub Actions Pipelines

On every PR ( `.github/workflows/ci.yml` ):

```

name: CI
on: [pull_request]
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - checkout
      - setup node / python
      - run: pnpm lint && ruff check backend/

```



```

build:
  needs: lint
  steps:
    - run: pnpm --filter frontend build
    - run: pnpm --filter extension build
test:
  needs: lint
  steps:
    - run: pytest backend/tests
    - run: pnpm vitest run

```

**On merge to `main`:** full test suite + Playwright E2E against an ephemeral environment, then auto-deploy to **staging**.

**On tagged release (`v*`):** deploy to **production** (manual approval gate).

## 15.2 Preview Deployments

- Vercel automatically generates a preview deployment per PR for `frontend/` — linked in the PR for reviewer QA.
- Backend preview environments (optional, Phase 8+): ephemeral Railway/Render environment per PR if capacity allows; otherwise staging serves as the shared pre-prod environment.

## 15.3 Production Deployment

- Manual approval required in GitHub Actions environment protection rules before a tagged release deploys to production.
- Blue/green or rolling deploy depending on host platform capability (Railway/Render support zero-downtime deploys natively).

## 15.4 Rollback Strategy

- Every production deploy is tied to a Git tag; rollback = redeploy the previous tag's container image.
- Database migrations are additive-first (avoid destructive migrations in the same release as risky features) so a code rollback doesn't require a DB rollback.
- Feature flags for any risky feature (e.g., new capture heuristic) allow disabling without a redeploy.

# 16. Versioning Strategy

Semantic Versioning ( `MAJOR.MINOR.PATCH` ).

| Stage             | Version Range                                    | Meaning                                      |
|-------------------|--------------------------------------------------|----------------------------------------------|
| Early development | <code>v0.1.0</code> - <code>v0.9.x</code>        | Pre-MVP, breaking changes expected freely    |
| MVP Alpha         | <code>v0.9.0</code>                              | Feature-complete internally, dogfood-only    |
| MVP Beta          | <code>v0.9.5</code>                              | Feature-complete, external testers (if any)  |
| Release Candidate | <code>v1.0.0-rc.1</code> , <code>rc.2</code> ... | Launch-candidate builds during Testing phase |

|                     |          |                                             |
|---------------------|----------|---------------------------------------------|
| Production Launch   | v1.0.0   | First public/production MVP release         |
| Post-launch patches | v1.0.x   | Bug fixes only                              |
| Post-MVP features   | v1.1.0 + | New functionality (Footprint, Shield, etc.) |

**Tagging:** annotated Git tags ( `git tag -a v1.0.0 -m "MVP launch"` ) pushed alongside the release; CI's production-deploy job triggers only on tag push matching `v*,*.*`.

# 17. Documentation Strategy

| Document                                       | Owner                                                | Update Cadence                                     |
|------------------------------------------------|------------------------------------------------------|----------------------------------------------------|
| README.md                                      | Whoever touches setup/build steps                    | Every time setup changes                           |
| Architecture docs<br>( docs/architecture/ )    | Tech lead                                            | End of each Phase                                  |
| API docs                                       | Auto-generated from FastAPI OpenAPI schema ( /docs ) | Automatic, on every deploy                         |
| Developer guide ( docs/contribution-guide.md ) | Whole team                                           | As conventions evolve                              |
| Setup guide ( docs/setup-guide.md )            | Phase 0 owner                                        | Locked after Phase 0, revisited if tooling changes |
| Changelog ( docs/changelog.md )                | Release owner                                        | Every tagged release                               |

All documentation lives in-repo ( docs/ ) so it version-controls alongside the code it describes — no external wiki as source of truth.

# 18. Deployment Roadmap

| Environment | Purpose                                    | Trigger                                  |
|-------------|--------------------------------------------|------------------------------------------|
| Development | Local machines, docker-compose             | Continuous, per-developer                |
| Staging     | Shared pre-prod, mirrors production config | Auto-deploy on merge to main             |
| Production  | Live MVP                                   | Manual-approved deploy on tagged release |

## Operational readiness:

- **Monitoring:** application performance monitoring on backend (e.g., request latency, error rate) via the hosting platform's built-in metrics or a lightweight APM.
- **Logging:** structured JSON logging from FastAPI, aggregated in the hosting platform's log viewer; extension logs captured to console + optional opt-in diagnostic upload.
- **Backups:** automated daily PostgreSQL backups (managed DB provider feature), retained per provider default (verify  $\geq 7$  days for MVP).
- **Health checks:** `/health` endpoint on backend polled by the hosting platform for auto-restart on failure.
- **Secrets management:** all secrets (LLM API key, DB credentials, JWT secret) stored in the platform's secret manager — never in the repo, never in client-side bundles.

## 19. Launch Checklist

---

- ☐ Backend deployed to production, `/health` returning 200
- ☐ Frontend deployed to production (Vercel), pointing at production API
- ☐ Extension built in production mode, package ready (Web Store submission or dev-mode dogfood build)
- ☐ Authentication flow verified end-to-end in production
- ☐ Capture pipeline verified against real webpage, PDF, and YouTube sources in production
- ☐ Never-capture exclusion rules verified against sensitive-content test matrix
- ☐ Memory Timeline displays real captured items correctly
- ☐ Semantic Search returns relevant results in production
- ☐ RAG Chat returns grounded answers with correct citations in production
- ☐ All P0/P1 bugs resolved; no open S0 issues
- ☐ Security checklist complete (auth bypass attempts, secrets audit, dependency vuln scan)
- ☐ Performance verified (search/chat latency within acceptable range under expected dogfood load)
- ☐ Monitoring, logging, and backups confirmed active in production
- ☐ Rollback procedure tested at least once (redeploy previous tag successfully)
- ☐ Documentation complete: README, setup guide, API docs, changelog for `v1.0.0`
- ☐ Version tagged `v1.0.0` in Git

## 20. Post-MVP Roadmap

---

Explicitly **not** part of this plan’s execution scope — tracked separately, to begin only after MVP launch and PRD update for next phase.

| Item                        | Notes                                                                                                                                             |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Footprint module            | Account/identity graph mapping — deferred per PRD; requires its own data model and consent flows                                                  |
| Shield module               | Risk detection/recommendations consuming Footprint + Memory — architecture intentionally left unresolved in PRD, needs definition before planning |
| Firefox Extension           | Port of Manifest V3 extension logic where compatible                                                                                              |
| Edge Extension              | Chromium-based, likely lowest-effort port after Chrome MVP                                                                                        |
| Mobile App                  | New platform, new architecture discussion required                                                                                                |
| Real-time Monitoring        | Live capture/alerting beyond MVP’s async pipeline                                                                                                 |
| Team Collaboration          | Multi-user shared Memory/Collections — significant permissions-model rework                                                                       |
| Enterprise Features         | SSO, admin controls, audit logs                                                                                                                   |
| Advanced Footprint Scanning | Deeper account/permission discovery beyond initial Footprint MVP                                                                                  |
| Advanced Shield             | Automated remediation actions, not just recommendations                                                                                           |
| Streaming AI Responses      | Token-streaming chat UX — deferred to keep MVP RAG implementation simpler                                                                         |

## 21. Month-by-Month Roadmap

---

## August — Foundations

---

- Week 1: Phase 0 (environment, repo, CI skeleton)
- Weeks 2-3: Phase 1 (Authentication)
- Weeks 3-4: Phase 2 (Extension skeleton)
- **Milestone:** Team can log in on web and extension against a shared backend.

## September — Core Data Loop

---

- Weeks 5-7: Phase 3 (Capture Pipeline)
- Weeks 5-8: Phase 4 (Backend Core APIs) — parallel track
- **Milestone:** First real capture flows extension → backend → stored Memory item.

## October — Visibility & Discoverability

---

- Weeks 8-10: Phase 5 (Frontend Dashboard)
- Weeks 10-12: Phase 6 (Search) begins
- **Milestone:** Users can see their Memory Timeline and semantically search it.

## November — Intelligence Layer

---

- Weeks 12-15: Phase 7 (AI / RAG Chat)
- Late November: Phase 8 (Testing) begins
- **Milestone:** RAG Chat answers grounded questions with citations; feature-complete MVP.

## December — Hardening & Launch

---

- Weeks 15-17: Phase 8 (Testing) completes — security, performance, regression
- Weeks 17-18: Phase 9 (Deployment & Launch)
- **Milestone:** `v1.0.0` tagged and live in production — **MVP Launched.**

## Appendix: Document Control

---

| Field               | Value                                                                                                                    |
|---------------------|--------------------------------------------------------------------------------------------------------------------------|
| Document            | Sentiora Development & Implementation Plan                                                                               |
| Version             | v1.0                                                                                                                     |
| Status              | Active — engineering execution reference                                                                                 |
| Companion Documents | PRD v2.1 FINAL, SRS, UI/UX Spec, Backend Technical Spec, API Spec, Database Schema (all frozen, referenced not repeated) |
| Owner               | Engineering Lead                                                                                                         |
| Review Cadence      | End of each Phase (Section 5)                                                                                            |